

mySCDN - FlashCacheCDN

Iancu Stefan-Teodor

Facultatea de Informatică, Universitatea Alexandru Ioan Cuza Iasi, Romania

1 Introducere

1.1 Viziune generala

Se propune realizarea unui sistem tip CDN (Content Delivery Network) prin care se doreste posibilitatea de a furniza un serviciu sau de a transfera date cat mai rapid. Latenta comunicarii va fi cat se poate de scazuta, iar utilizatorul va avea parte de o experienta buna cu aplicatia. Odata cu aceasta, imi propun ca realizarea acestui proiect (achizitia hardware-ului, a serviciilor de deployment etc. ce vor putea face posibila existenta intregului sistem) sa genereze costuri minime si sa ofere un serviciu software de calitate. Solutia proiectului propune distributia mai multor noduri in locatii geografice diferite care vor putea servi clientii in mod optimizat.

1.2 Obiective

Proiectul isi propune urmatoarele obiective:

- Se pune pe primul loc experienta clientului.
- Alocarea celui mai convenabil nod pentru fiecare client in parte.
- Evitarea riscului de a compromite tot serviciul si aplicatia, avand mereu o varianta de backup (in caz ca un nod va avea parte de crash, folosim alt nod ce va overi un serviciul cu o latenta pertinenta clientului).
- Gestionarea concurentei in mod cat mai fiabil.
- Optimizarea software-ului pentru a solicita cat mai putin CPU.
- Optimizarea in timp a sistemului de caching, cat si a resurselor generate de concurenta.
- Evitarea cazurilor neprevazute si tratarea erorilor.

2 Tehnologii Aplicate

- La nivelul transport, proiectul se va folosi de protocolul TCP in intregul sistem. Chit ca este mai costisitor din punct de vedere al dimensiunii pachetului si al timpului de transmisie al unui schimb de mesaje valid, aduce certitudinea ca niciun pachet nu se va pierde sau va fi corupt (prin mijloace

interne). Proiectul nu isi permite, spre exemplu, alocarea eronata a unui nod din America pentru un client din Madagascar prin coruperea a 3 bytes a unui pachet ce contine adresa IPv4 a nodului.

- Pentru a stoca informatii despre cache-ul disponibil pe fiecare nod proiectul se va folosi de formatul JSON pentru a putea deserializa continutul in obiecte stocate in memorie. Fisierele de configurare a serverelor si a clientului vor fi de tip `.ini` pentru ca sunt lightweight (la fel si API-ul) si usor de parsat. In consecinta, am folosit doua API-uri open-source pentru limbajul C++ ce se vor gasi in notele bibliografice.
- Arhitectura este proiectata pentru sisteme Unix-based. La nivelul comunicarii in retea m-am folosit de bibliotecile standard din C/C++ pentru Unix sockets.
- Din punct de vedere al concurentei, serverele vor implementa multithreading, multiprocessing si multiplexing in functie de caz, asa ca am folosit biibliotecile standard `<thread>` din C++ si `<unistd.h>` din C pentru a le gestiona.
- Se vor utiliza structurile de date oferite de biblioteca standard din C++ si paradigma OOP pentru a realiza logica aplicatiei.
- Cmake si bash scripts pentru build usor si instalarea dependintelor.

3 Structura Aplicației

Arhitectura aplicatiei se va baza pe 4 entitati: - Main server - Edge server - EDNS server - Client

O descriere pe scurt pentru fiecare:

- Main server-ul va fi responsabil de stocarea tuturor resurselor originale si va oferi updates catre edge severe, va tine evidenta cache-ului si a load-ului de pe edges si o va transmite serverului autorativ EDNS.
- Edge serverele vor tine in cache anumite resurse pe care le va servi clientilor, in timp ce transmite informatii despre load la main server.
- EDNS server-ul va primi informatiile despre toate edge serverele si le va tine mapate pentru a putea face o decizie in functie de cache, load si locatia geografica. (va juca si rol de load balancer)
- Clientul va face un request cu resursa dorita catre serverul EDNS, va primi raspuns o adresa IP si se va conecta catre edge serverul cu IP-ul respectiv.

Intr-un mod foarte general, arhitectura acestui gen de aplicatie poate fi descrisa foarte bine printr-o imagine:

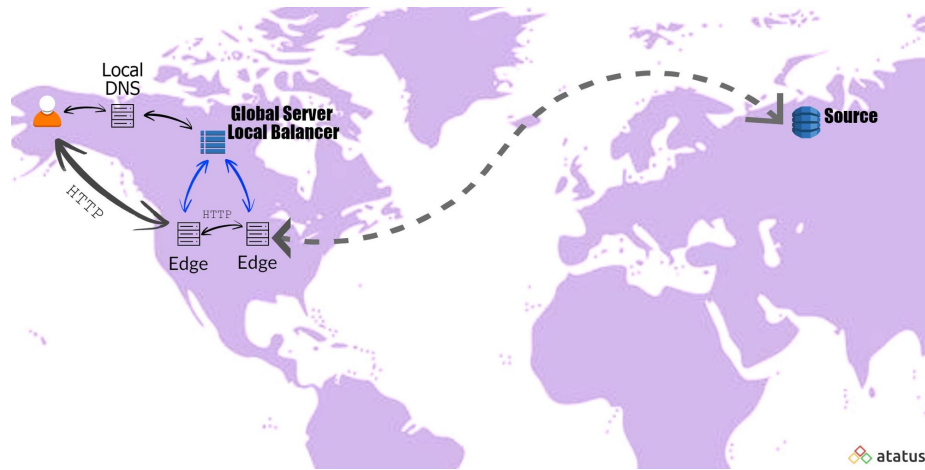


Fig. 1. CDN over the world

Iar acum sa vedem următoarea diagrama (zoom in):

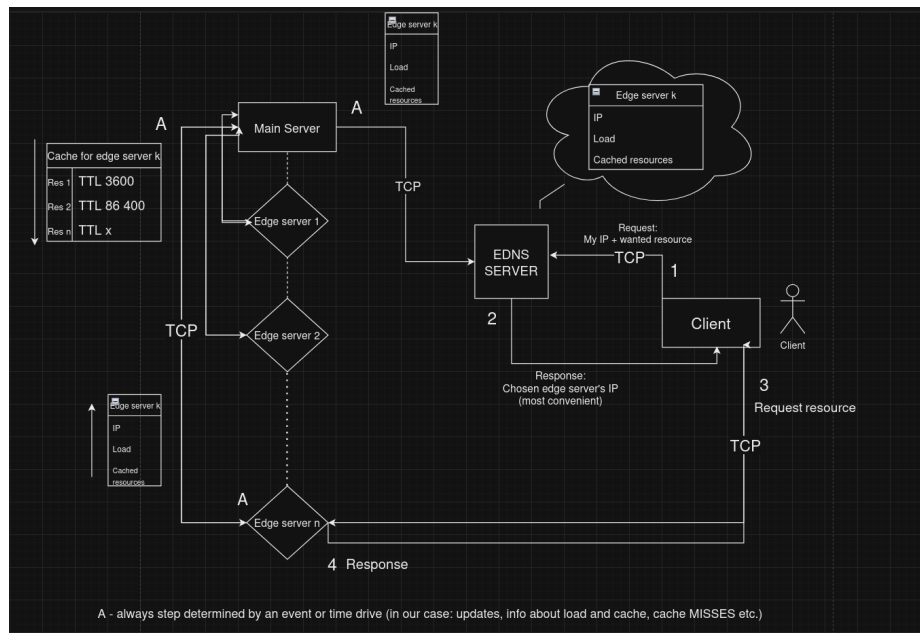


Fig. 2. Arhitectura CDN

4 Aspecte de Implementare

4.1 Protocolul la nivelul aplicației și secțiuni de cod inovative

Protocolul Client-EDNS server:

- Pachetul request va fi format din: lungimea pachetului + IP-ul clientului + resursa + null byte
- Raspunsul va fi 4 bytes adresa IP a edge server-ului

```
// Creating the packet
const size_t dns_packet_length = sizeof(size_t) + resource.length() + 1 + sizeof(in_addr_t);
// length header (8 bytes on x64 arch) + sender's public IPv4 (4 bytes) + len(request) + nullptr
char* dns_packet = new char[dns_packet_length];
const in_addr_t pub_ipv4n_addr = inet_addr(ep:public_ip.c_str());
memcpy(dns_packet, &dns_packet_length, sizeof(size_t));
memcpy(dest: dns_packet + sizeof(size_t), src: resource.c_str(), resource.length() + 1);
memcpy(dest: dns_packet + sizeof(size_t) + resource.length() + 1, &pub_ipv4n_addr, sizeof(in_addr_t));

// Sending the packet to the EDNS0 server
size_t bytes_sent = send(socket_fd, dns_packet, dns_packet_length, flags: 0);
```

Fig. 3. Ex. pachet Client-EDNS server

Protocolul Client-Edge server:

- Pachetul request va fi format din: lungimea pachetului + resursa + null byte
- Raspunsul va fi lungimea pachetului + raspunsul + null byte
- *Asemanator cu Fig.3*

Protocolul Edge server-Main server:

- Edge serverul va transmite periodic informatii despre el printr-un transfer de fisier JSON sau va transmite un GET request catre server pentru a cere o resursa in cazul unui cache MISS
- Main serverul va transmite anumite resurse catre edge server, impreuna cu o cuanta TTL (time to live)
- *Asemanator cu Fig.3* dar transferul de fisiere va fi impartit in byte chunks

Protocolul Main server-EDNS server:

- Din Main server se va transmite fisierul JSON cu datele despre cache, load de la edge-servers
- *Asemanator cu Fig.3* dar transferul de fisiere va fi impartit in byte chunks

```
{
  "10.126.86.215" : {
    "name": "edge-server-3",
    "load" : {
      "cpu_load": [
        0.22,
        0.05,
        0.02
      ],
      "memory_load": [
        376.5,
        900.0
      ],
      "disk_load": [
        1600.2,
        6384.6
      ]
    },
    "resources" : [". /test", ". /test_5_seconds"]
  }
}
```

Fig. 4. JSON data

```

class Cache
{
    string main_server_ip;
    int number_of_edge_servers;
    vector<Edge_server> edge_servers;
    void initialize(); // Initializing the cached data from the JSON config file
public:
    Cache(); // Will call initialize after allocating memory
    void update_mapping();

    // It is needed in order for the thread to access the actual object's instance function update_mapping()
    static void* update_mapping_entry_point(void* actual_object);

    // Decide which edge-server will pick based on the implemented algorithms
    in_addr_t decide(const char* resource, in_addr_t client_ip);
};

```

Fig. 5. Deserializat nivel 1

```

class Edge_server
{
    Load load;
    in_addr_t ip_address;
    string name;
    unordered_set<string> resources; // 0(1) lookups by using unordered_set
    // Compute the average load of the machine
    float get_average_load() const;

    // Estimating the geographical distance between the client and the edge-server
    float get_geo_distance_estimation_index(in_addr_t client_ip);
public:
    Edge_server(const Load& load, in_addr_t ip_address, const string& name, const unordered_set<string>& resources); // Constructor

    // Computing the final grade of the edge-server based on cached content, load and geolocation
    float compute_decide_grade(in_addr_t client_ip, const char* resource);
    string get_name() const;
    in_addr_t get_ip() const;
};

```

Fig. 6. Deserializat nivel 2

4.2 Concurenta

- **Main server** - la pornire va face **pre-forking** pentru a gestiona cele N edge server, iar procesul radacina va comunica cu serverul EDNS. Am ales aceasta abordarea deoarece proiectul nu isi permite ca serverul Main sa cada in cazul in care un proces da crash. Chiar dac a procesul de forking este costisitor, acesta se face la initializare si nu afecteaza comunicarea concurenta.
- **Edge server** - va contine 2 procese. Unul va comunica cu Main serverul, iar unul va crea un **thread pool** de o dimenisune k (care se va imbunatati in

acuratete in timp). Fiecare thread ruleaza aceeaasi functie pentru a servi clientul, in care apeleaza primitiva `accept()` in interiorul unui `mutex.lock()`. Folosesc aceasta abordare deoarece pentru un volum mare de trafic crearea unui thread pe moment poate costa cateva milisecunde + solicitare CPU, desi thread-urile sunt lightweight. Aici va depinde si de serviciul folosit pentru deployment daca accepta **multithreading**.

- **EDNS server** - Va avea 2 threaduri ce vor avea acces simultan la acea variabila **Cache** alocata pe HEAP. Unul o va modifica cand primeste updates de la Main server, iar unul va trata clientii in mod **multiplexat** si va accesa variabila pentru a face decizia de alegere a edge serverului cel mai convenabil. Bineinteles, ne vom folosi de `mutex.lock()` pentru a ne proteja de *undefined behavior* (un thread modifica, iar unul citeste). Am folosit aceasta abordare cu thread-uri pentru a putea avea acces la **Cache** simultan, iar multiplexarea se justifica prin reducerea costului (a call-urilor de CPU) si prin faptul ca serviciul oferit de un server DNS este rapid si light.

- **Client** - 2 procese. Unul va primi INPUT si va face request-uri catre EDNS server, apoi creeaza un nou proces apeland `fork()` si redirecteaza request-ul catre edge server, apoi asteptand OUTPUT. Am ales aceasta abordare pentru a face posibil ca un request sa nu fie limitat de timpul de procesare al requestului anterior (sau anterioare).

In concluzie, toata arhitectura sistemului CDN dezvoltat poate fi encapsulat in urmatoarea diagrama detaliata (zoom in):

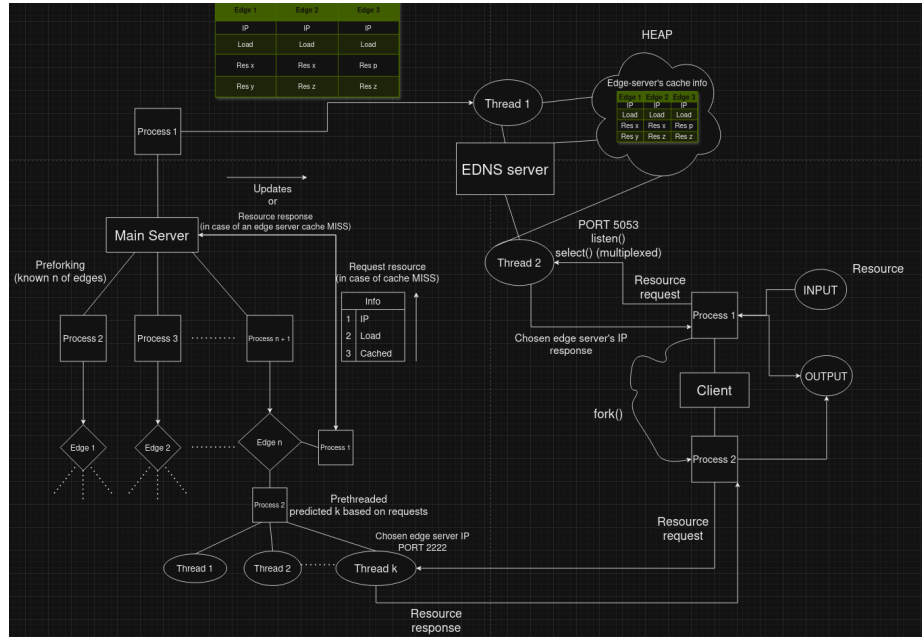


Fig. 7. Arhitectura detaliata

4.3 Scenariu real de utilizare

O firma de aviatie vrea sa isi distribuie serviciul online de booking catre toata planeta, deoarece este populara si detine multe avioane. Firma are 2 sedii unde tine serverele, unul in Europa si unul in USA. Analistii de date angajati la firma si-au dat seama ca desi clientii ce calatoresc in zona Asiei, Africii sau a Americii de Sud sunt foarte multumiti de servicii, nu genereaza asa mult profit. In urma altui studiu, si-au dat seama ca persoanele la care pagina web se incarca in minim 5s au o probabilitate scazuta sa cumpere biletul de avion. In consecinta, firma isi face deploy serviciului web pe edge servere plasate in zone geografice strategice pe toata planeta, furnizate de o alta firma contra-cost. In lunile urmatoare, profitul deja a inceput sa creasca. Totodata au redus incarcatura serverelor din sediu, deoarece acum tot traficul de pe site-ul web este distribuit in mod optimizat si au redus riscul de DoS si DDoS.

5 Potențiale îmbunătățiri:

- Protectie anti DoS si DDoS, proxy rate limiter pentru serverul EDNS
- Pe langa TTL, implementarea unui algoritm de caching in edge server, precum LFU, LRU etc.

- Mecanism de caching pentru lookups pentru serverul EDNS.
- Dezvoltarea unui tunel de encriptie peste comunicari intre servere, precum TLS.

6 Referințe Bibliografice

- <https://edu.info.uaic.ro/computer-networks/>
- <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
- <https://aws.amazon.com/what-is/cdn/>
- [https://en.wikipedia.org/wiki/Multithreading_\(computer_architecture\)](https://en.wikipedia.org/wiki/Multithreading_(computer_architecture))
- <https://www.youtube.com/watch?v=pdPSLm629yk&t=454s>
- https://en.wikipedia.org/wiki/Content_delivery_network
- <https://en.cppreference.com/w/cpp/thread/thread>
- <https://en.cppreference.com/w/cpp/thread/mutex>
- https://en.cppreference.com/w/cpp/container/unordered_set
- <https://github.com/nlohmann/json>
- <https://github.com/dzilles/configparser>